Lab No 9:

Programs on Matrix using CUDA

Objectives:

In this lab, students will be able to

1. Understand how to write kernel code in CUDA to perform operations on matrix
2. Learn about CUDA 2D blocks/threads
3. Write simple programs on two dimensional arrays
4. Learn how to represent sparse matrix using CSR format and perform parallel operations using it.

Solved Exercise:

Implement a program in CUDA to find transpose of a matrix in parallel.

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void transpose(int *a, int *t)
        {
        int n=threadIdx.x, m=blockIdx.x, size=blockDim.x, size1=gridDim.x;
        t[n*size1+m]=a[m*size+n];
        }
int main(void)
        {
        int *a,*t, m,n,i,j;
        int *d_a,*d_t;
        printf("Enter the value of m: ");scanf("%d",&m);
        printf("Enter the value of n: ");scanf("%d",&n);
        int size=sizeof(int)*m*n;
        a=(int*)malloc(m*n*sizeof(int));
        c=(int*)malloc(m*n*sizeof(int));
        printf("Enter input matrix:\n");
        for(i=0;i<m*n;i++)
                scanf("%d",&a[i]);

        cudaMalloc((void**)&d_a,size);
```

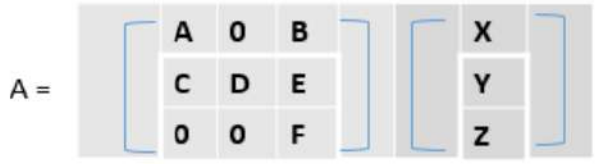
```
cudaMalloc((void**)&d_t,size);

cudaMemcpy(d_a,a,size,cudaMemcpyHostToDevice);
transpose<<<m,n>>>(d_a,d_t);
cudaMemcpy(t,d_t,size,cudaMemcpyDeviceToHost);
printf("Result vector is:\n");
for(i=0;i<n;i++)
        {
        for(j=0;j<m;j++)
                printf("%d\t",t[i*m+j]);
        printf("\n");
        }

getchar();
cudaFree(d_a);
cudaFree(d_t);
return 0;
}
```

Compressed Sparse Row (CSR) format:

- A sparse matrix is a matrix where the majority of the elements are zero. Sparse matrices are stored in a format that avoids storing zero elements.
- The compressed sparse row (CSR) format is a popular, general-purpose sparse matrix rep- resentation. CSR explicitly stores column indices and nonzero values in arrays col_index and data as shown below.
- A third array of row pointers, row_ptr, give the starting location of every row in the compressed storage.



i =	0	1	2	3	4	5	
Data[i]	[A	B	C	D	E	F]	=non zero elements
col_index	[0	2	0	1	2	1]	=column indices of elements
Row_ptr	[0	2	5]				=pointers to the first element in each row

Lab Exercises:

1. Implement a program in CUDA to add two Matrices for the following specifications:
   a. Each row of resultant matrix to be computed by one thread.
   b. Each column of resultant matrix to be computed by one thread.
   c. Each element of resultant matrix to be computed by one thread.

2. Implement a program in CUDA to multiply two Matrices for the following specifications:
   a. Each row of resultant matrix to be computed by one thread.
   b. Each column of resultant matrix to be computed by one thread.
   c. Each element of resultant matrix to be computed by one thread.
3. Implement a program in CUDA to perform parallel Sparse Matrix - Vector multiplication using com- pressed sparse row (CSR) storage format. Represent the input sparse matrix in CSR format in the host code.
4. Implement a program in CUDA to read MXN matrix A and replace 1st row of this matrix with the same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.

Additional Exercises:

1. Implement a CUDA program that reads a MXN matrix A and produces a resultant matrix B of same size as follows: Replace all the even numbered matrix elements with their row sum and odd numbered matrix elements with their column sum.

Example:

	A				B		
I/p:	**1**	**2**	**3**	O/p:	5	**6**	9
	4	5	6		15	7	15

2. Implement a CUDA program to read a matrix A of size N X N. From the given matrix A, replace the principal diagonal elements with zero, elements above the principal diagonal by their factorial and elements below the principal diagonal by their sum of their digits.

3. Implement a CUDA program that reads a character type matrix A and integer type matrix B of size MXN. It produces an output string STR such that, every character of A is repeated r times (where r is the integer value in matrix B which has the same index as that of the character taken in A). Write the kernel such that every value of input matrix must be produced required number of times by one thread.

Example:

A				B			
p	C	a	P	1	2	4	3
e	X	a	M	2	4	3	2

Output String STR: pCCaaaaPPPeeXXXXaaaMM